Lecturer: Andrew Parkes
http://www.cs.nott.ac.uk/~pszajp/

COMP2054 ADE

Lec17

Binary Search Trees

Motivations

- Suppose you have an array A (of ints) and want to search for a particular int k
- If the array is unsorted then no choice but to scan all elements, hence O(n)
- If it is sorted then we can do a lot better. How?

Binary search in a sorted array

Search for index of x in a sorted (sub)array

```
int search (int x, int[] A, int L, int R)
   if ( R < L ) return -1 // "fail"
   int M = (L+R)/2
   if (x < A[M] ) return search(x, A, L,M-1)
   if (A[M] = x) return M
   if (x > A[M] ) return search(x, A, M+1,R)

int search (int x, int[] A)
   return search(x,A,0,A.size-1)
```

Uses standard trick of extending the code to search a sub-array, and so allow a recursion

Binary search complexity

- Using recurrence relations
  - T(n) = T(n/2) + 1
    - "n/2" because only have to search ½ the array after recursion
    - "1" as the cost of doing the comparison and deciding whether to go left or right
  - Can do exactly (exercise!)

- Use Master Theorem (revise lectures if needed):
  - a=1, b=2, log_b(a) = log_2(1) = 0
  - So c = log_b(a) = 0
  - Have f(n) = 1 which is Θ(n^0)
  - So is Case 2: and then is n^0 but "get an extra log"
  - Hence, Θ(log(n))

Motivations

- Binary Search for an element within a sorted array is fast
  - The array to be searched is halved at each iteration
  - Hence O(log n) (Previous slide also proved this).
- It only works because of the step of
  "knowing whether to go left or right"
- But arrays suffer from being slow
  - E.g. O(n), to insert new elements; because need to shift O(n) elements to make room for them
- **A "Binary Search Tree" attempts to**
  - **fix the inefficiency of insertion whilst**
  - **keeping good O(log n) properties of binary search**
- **We need a tree in which we always "know which way to go when looking for an element":**

Defn: Binary Search Tree

A binary search tree

- Is a binary tree storing key-value entries in the nodes
- Satisfies a "search tree" property:
  - For every node M, and

    For any node L in the **LEFT SUBTREE** of M, and

    For any node R in the **RIGHT SUBTREE** of M
  - We must have

    key(L) <= key(M) <= key(R)
  - Or since we exclude duplicate keys we must have

    key(L) < key(M) < key(R)

BST property

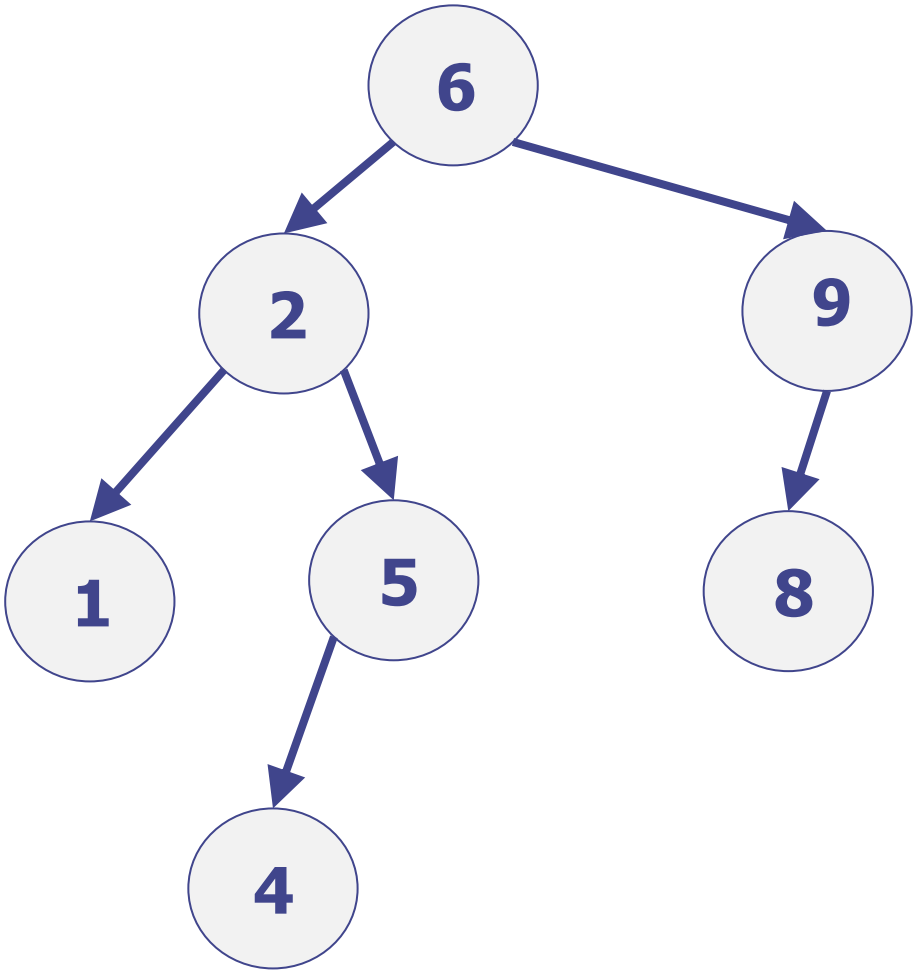
For every node in BST:

- ALL the keys in its left **subtree** must be strictly smaller, and
- ALL the keys in its right **subtree** must be strictly larger

BST Examples: Good BST

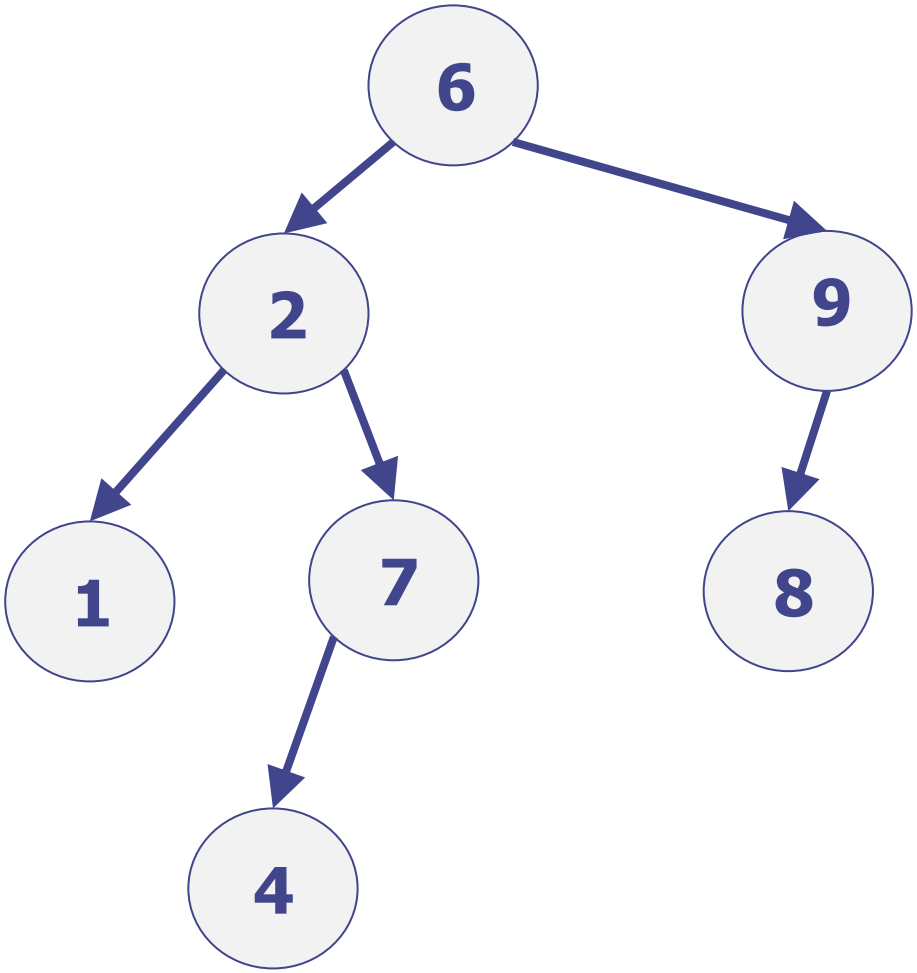
- This is a valid BST:

- For node 6
  - Left= 1,2,4,5
  - Right= 8,9
- etc.



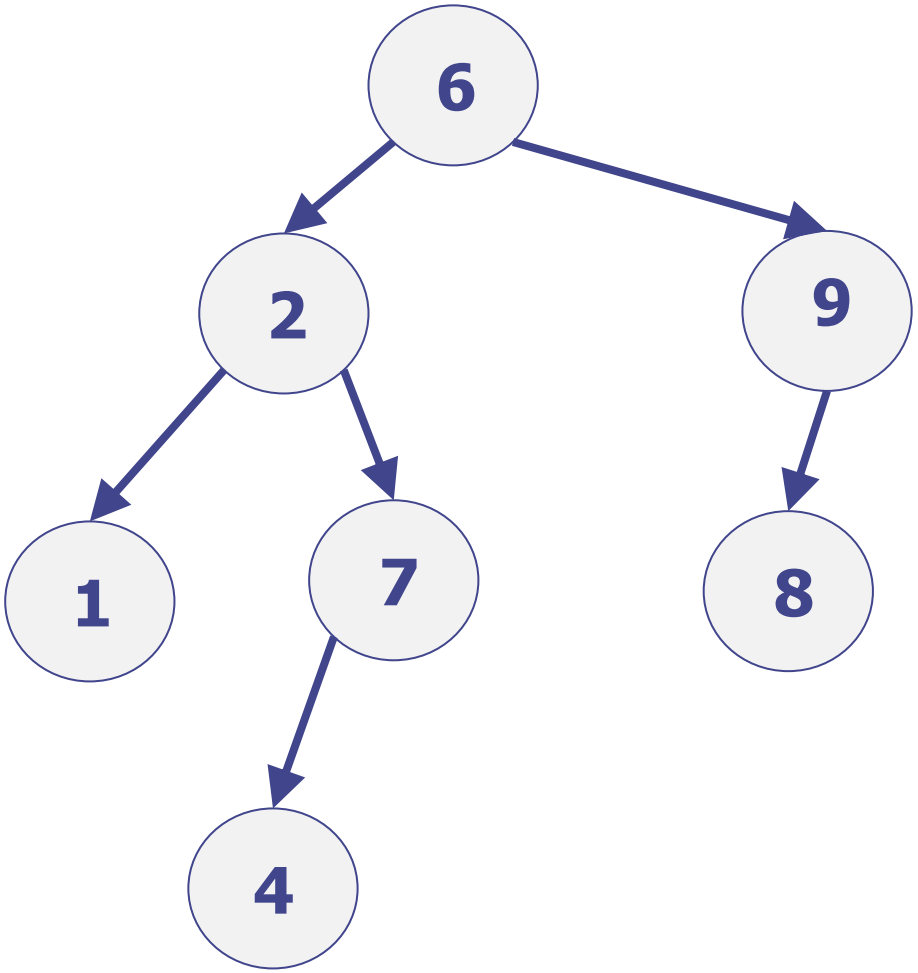
BST Examples: Good BST ??

- Is this a BST?



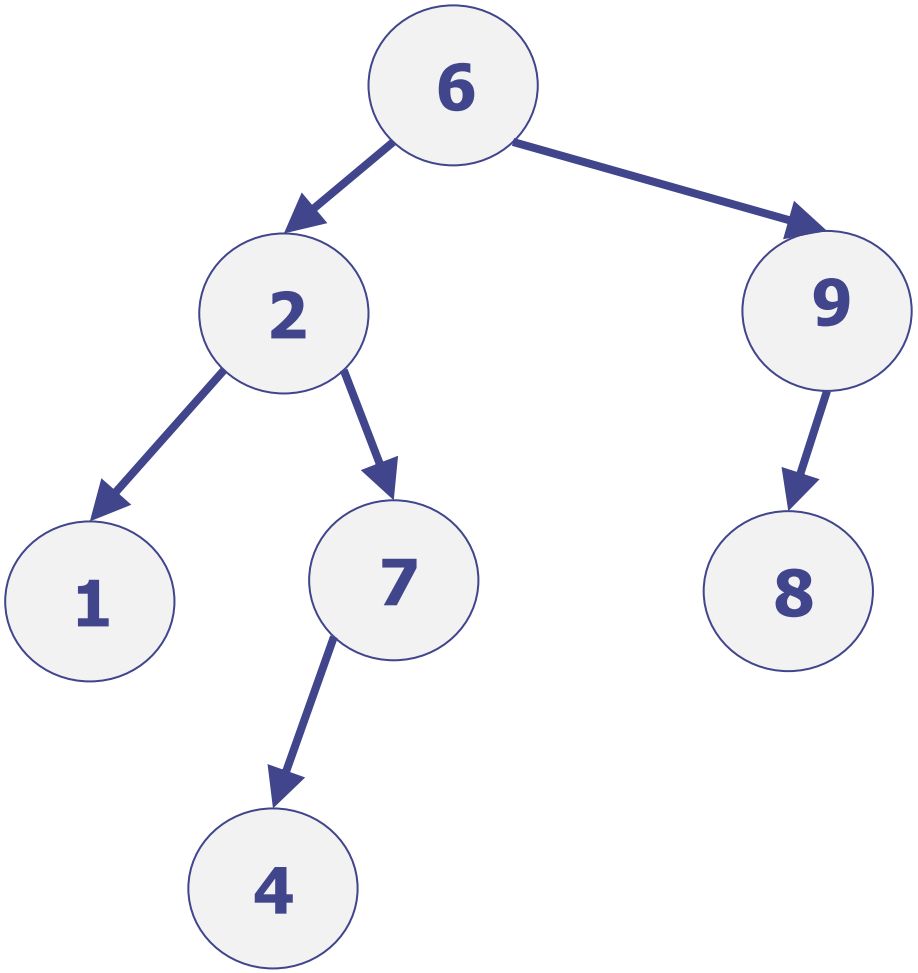
BST Examples: Good BST ??

- Is this a BST?
- "*Yes, because for every node: the left child is smaller; and the right child is bigger*"
- ??????



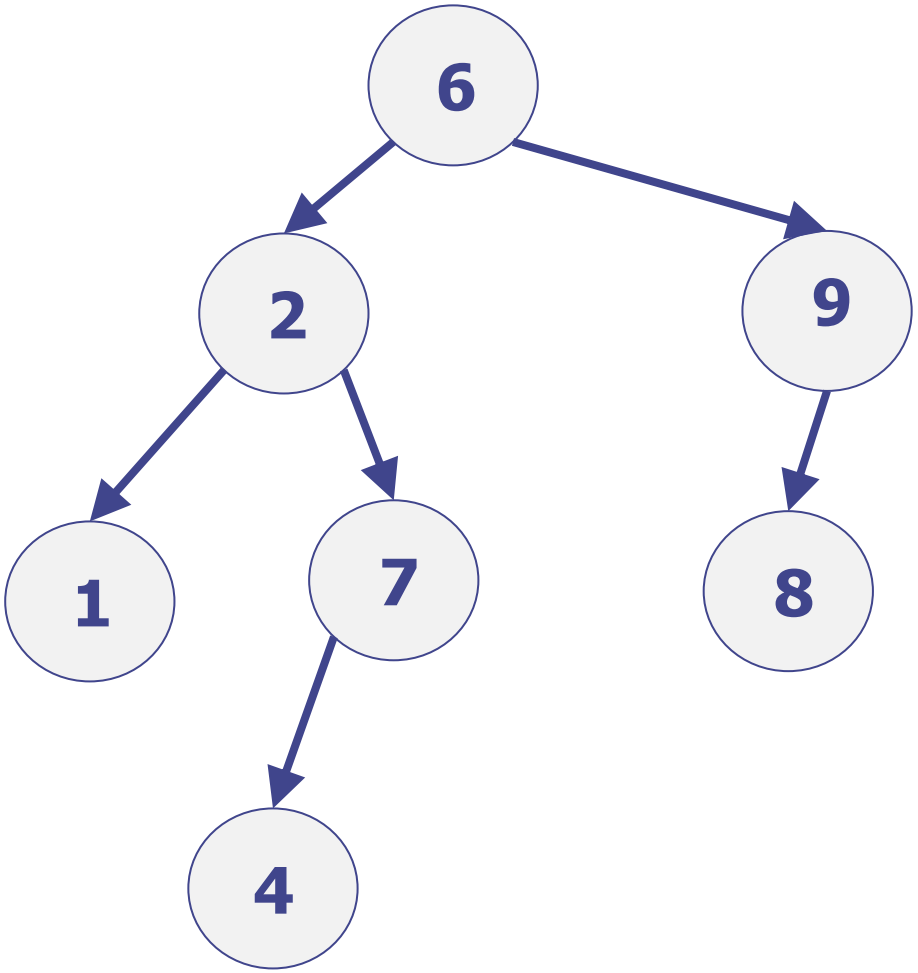
BST Examples: Good BST ??

- "Yes, because for every node:
  the left child is smaller;
  and
  the right child is bigger"
- IS WRONG !!!
- Just looking at the children is not sufficient



BST Examples: Good BST ??

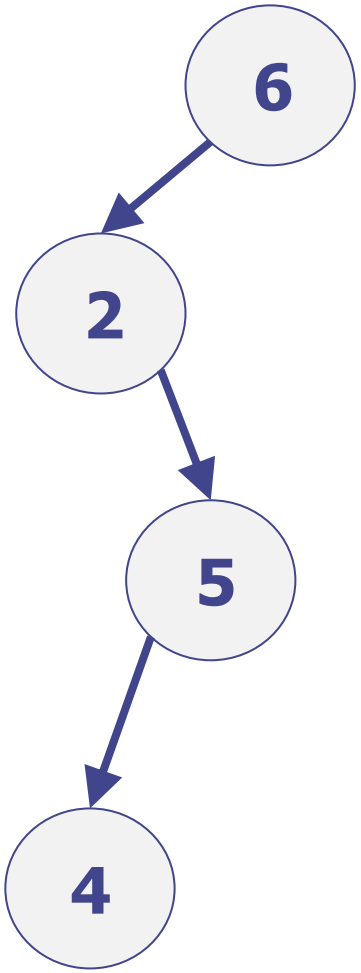
- It is not a BST !
- We need the subtree property:
  - Left subtree of 6 contains 7 which is bigger – so it fails
- Though note the subtree with root 2 is a BST; so both children of 6 are BSTs themself



BST Examples: Good BST

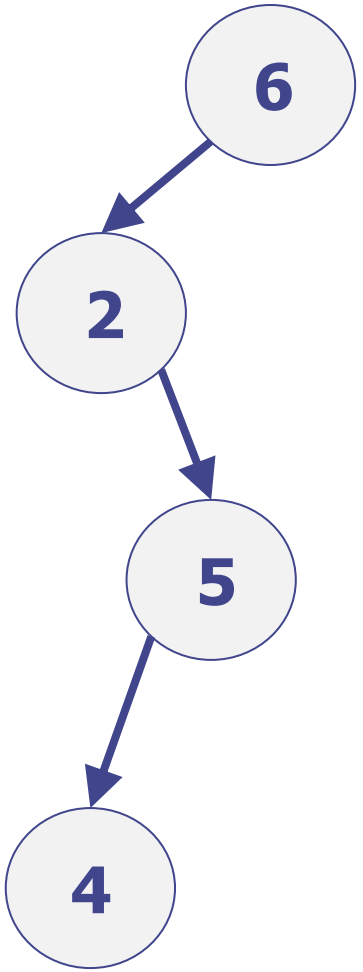
- Is this a BST?

- "*No, because it is not binary*"
- ?????



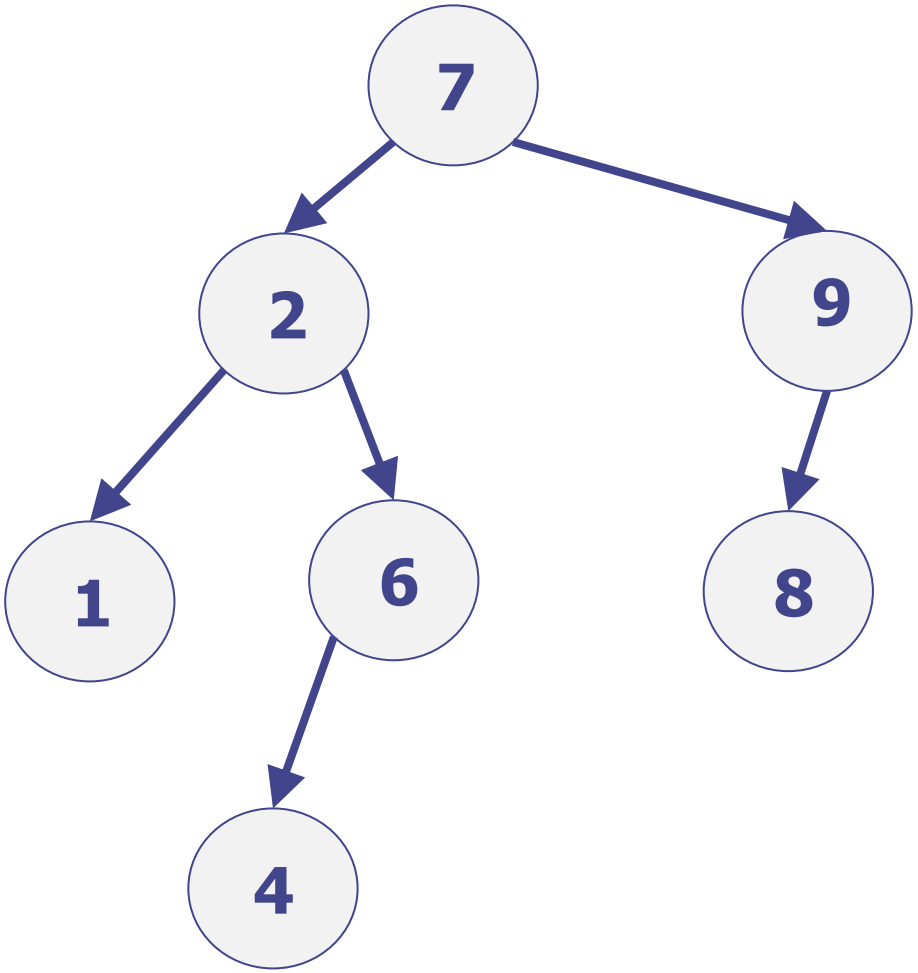
BST Examples: Good BST

- This is a valid BST !!
- "No, because it is not binary" is wrong
- "binary" just means "at most two children", and does not imply "some node must have two children"
- A BST can be just a long chain:
  - The height will then be O(n)
  - This is important later



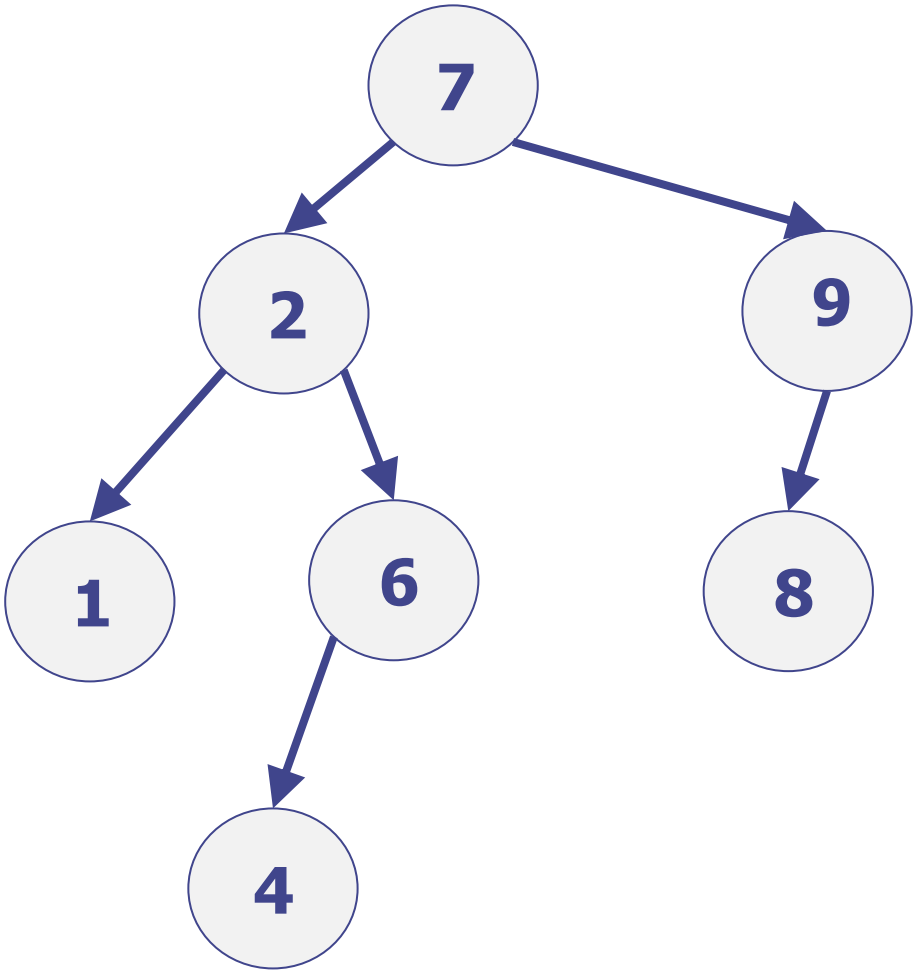
BST Property : In-order traversal

- In previous lecture consider in-order traversals
  - Visit left sub-tree
  - Visit node
  - Visit right sub-tree
- What do get from in-order traversal of a BST ?



BST Property : In-order traversal

- In-order traversal of a BST ?
- This gives the keys in a sorted order
- E.g. 1,2,4,6,7,8,9
- If try it on a "not a BST" then the order is not sorted
- Exercise: try it!



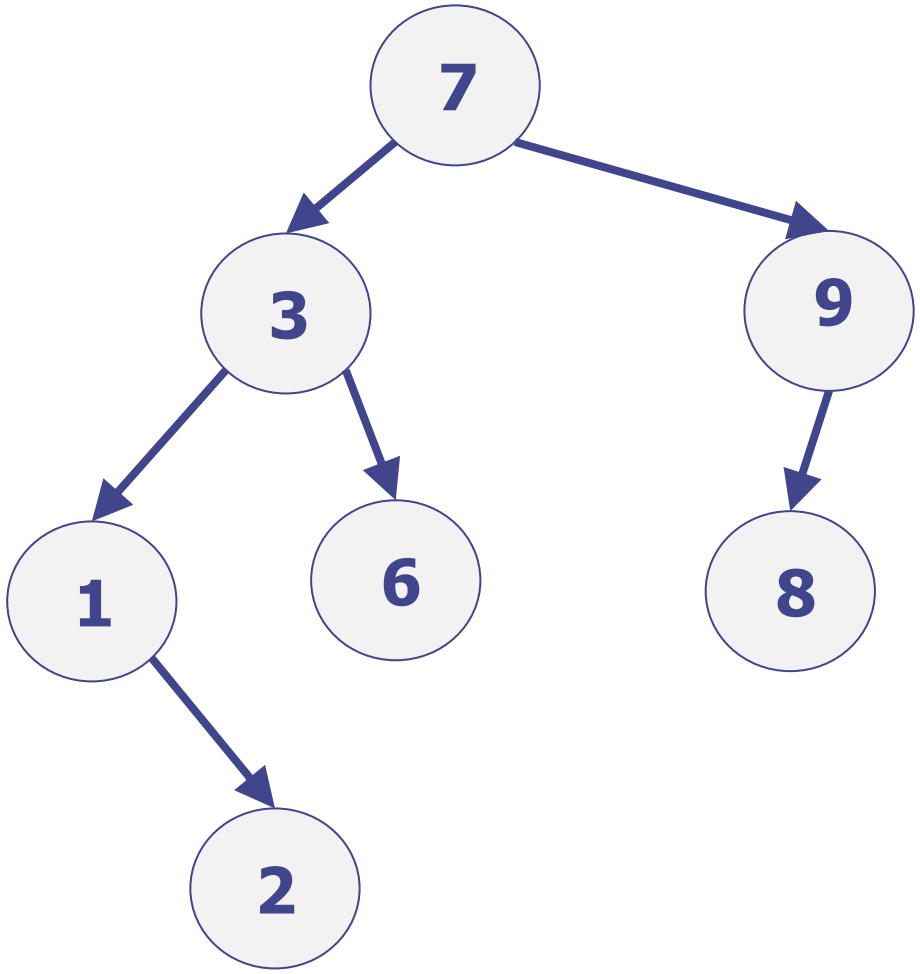
Fundamental Property of BST

Essentially, directly from the definition:

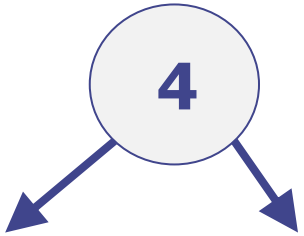
- An in-order traversal of a Binary Search Trees visits the keys in sorted (increasing) order

BST Property : Get the minimum?

- How do we find the minimum key in a BST?
- If there is a left child then it must have a smaller key
- Hence, repeatedly take the left child until there no longer is a left child
- Note a right child is bigger so is ignored
- Example: find node 1
- Cost is just O(height)
- Obviously: Max is just to always take right child.



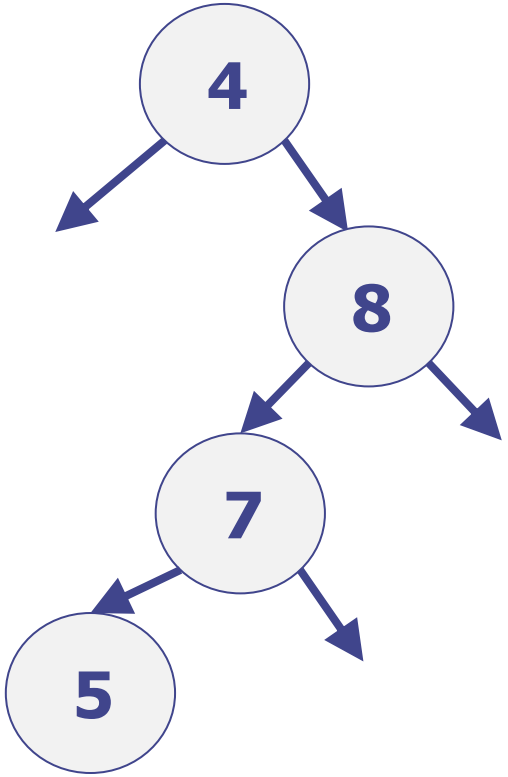
BST: Searching for a value



- Suppose we need to get(5)
  - Is there are node with key=5 ?
- Have to start with the root,
- Suppose the root has key=4
- What can we deduce?
  - Clearly, we will need to try the right subtree
  - But might we find the key=4 by moving to the left child of the root?
  - No, because 5 > 4
- If BST definition only had "left child must be smaller" then we could not exclude the left subtree
  - This is why "subtree" not "child" is vital in the definition of BST

BST: Searching for a value

- Suppose we need to get(5)
  - Is there are node with key=5 ?
  - The root has key=4
    - So, we must go to the right child
  - Now, we find 8, but 5 < 8, and so must try the left child
  - Now, we find 7, but 5 < 7, and so must try the left child
  - Now we find 5 and so succeed



- Followed a downward path from the root, to a successful node, or to a failed leaf node
- Worst-case length of the path, and so worst-case of get(k) is O(h), where h is the height of the tree
  - Note the non-followed branches could contain large trees, but we do not need to look at them

Search in a BST

Search for node containing x in a BST (sub)tree

```
int search (int x, BST T)
   if ( T == null ) return null // "fail"
   int M = T.root.key
   if ( x = M) return M
   if (x < M) return search(x, T.left)
   if (x > M ) return search(x, T.right)
```

Start using
int search (x, T.root)

Uses standard trick of extending the code to search a sub-structure, and so allow a recursion

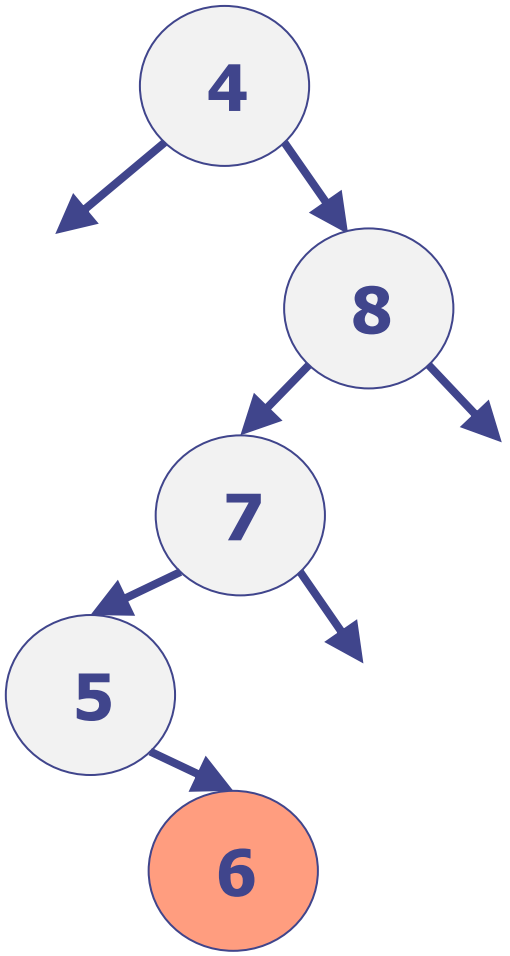
Recursive vs. Iterative

- Generally: Recursive programs are easier to implement, but less efficient
  - because of the overhead of a function call
- For best efficiency, will need to convert to an iterative program

  "while (test) { … }"
- Exercises (offline): convert to iterative
  - binary search of an array
  - search in a binary search tree

BST: Inserting a value: add(6)

- When inserting a new value, add(k), we must do it in such a way that get(k) would succeed
  - Or we need to overwrite the node if k is already present
- So, for add(k,v) just follow the process for get(k)
  - If find an existing node, then overwrite it with the new v value
  - Otherwise, we must have reached a node with no child in the correct place
    - So, create an new node with (k,v) and place it as the child
- Example from slide 24: get(6) would fail as 5 has no right child, so just add a new node with key 6

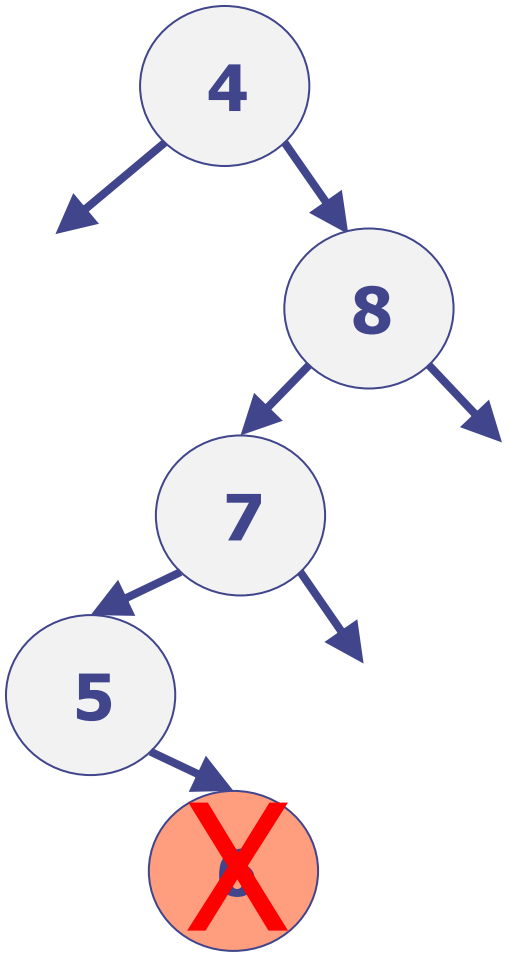


BST deletion : delete(k)

- We have seen that get(k) and add(k,b) were easy
- Deletion can be more difficult
- We start by finding the node we need to delete – by following get(k) process
- Then we have 3 cases:
  - The k-node has no children
  - The k-node has 1 child
  - The k-node has 2 children

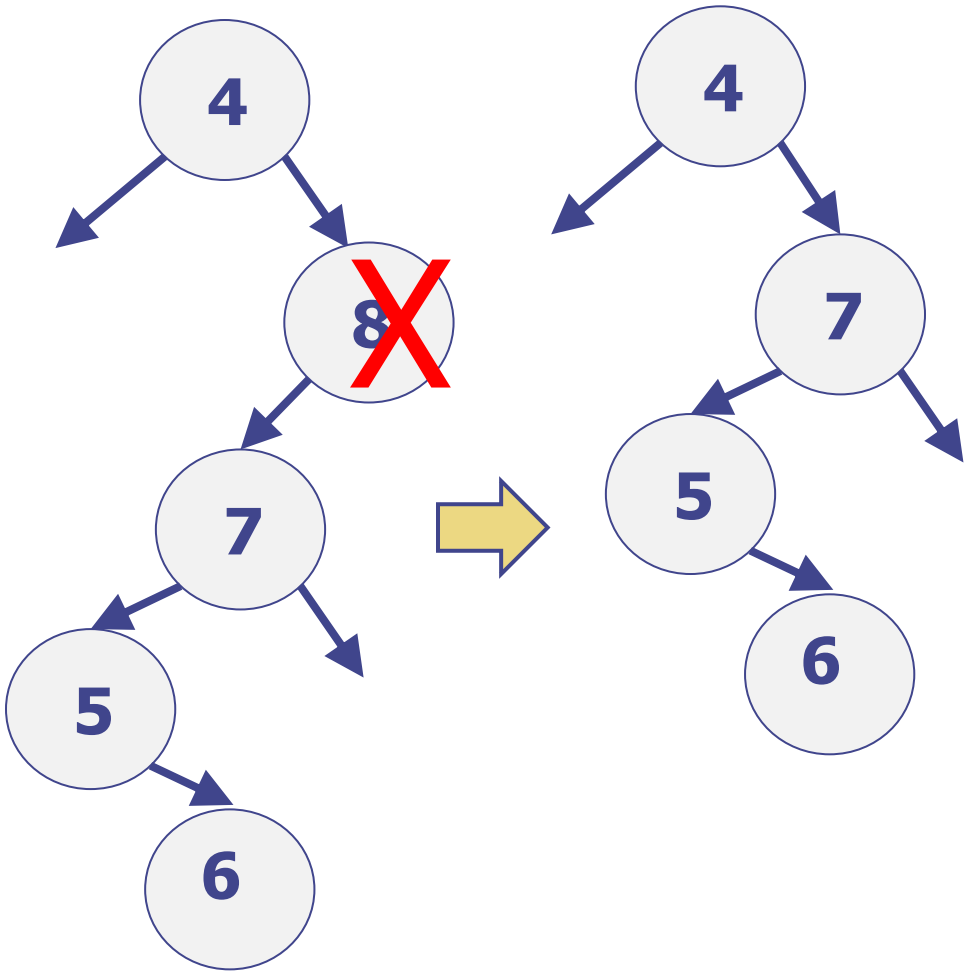
BST deletion : delete(k) : zero children

- Case: The k-node has no children
- This is trivial
  - we can just delete the node – disconnect it from the parent
- Example: delete(6) will just delete the node 6 and set the right child of 5 to be null



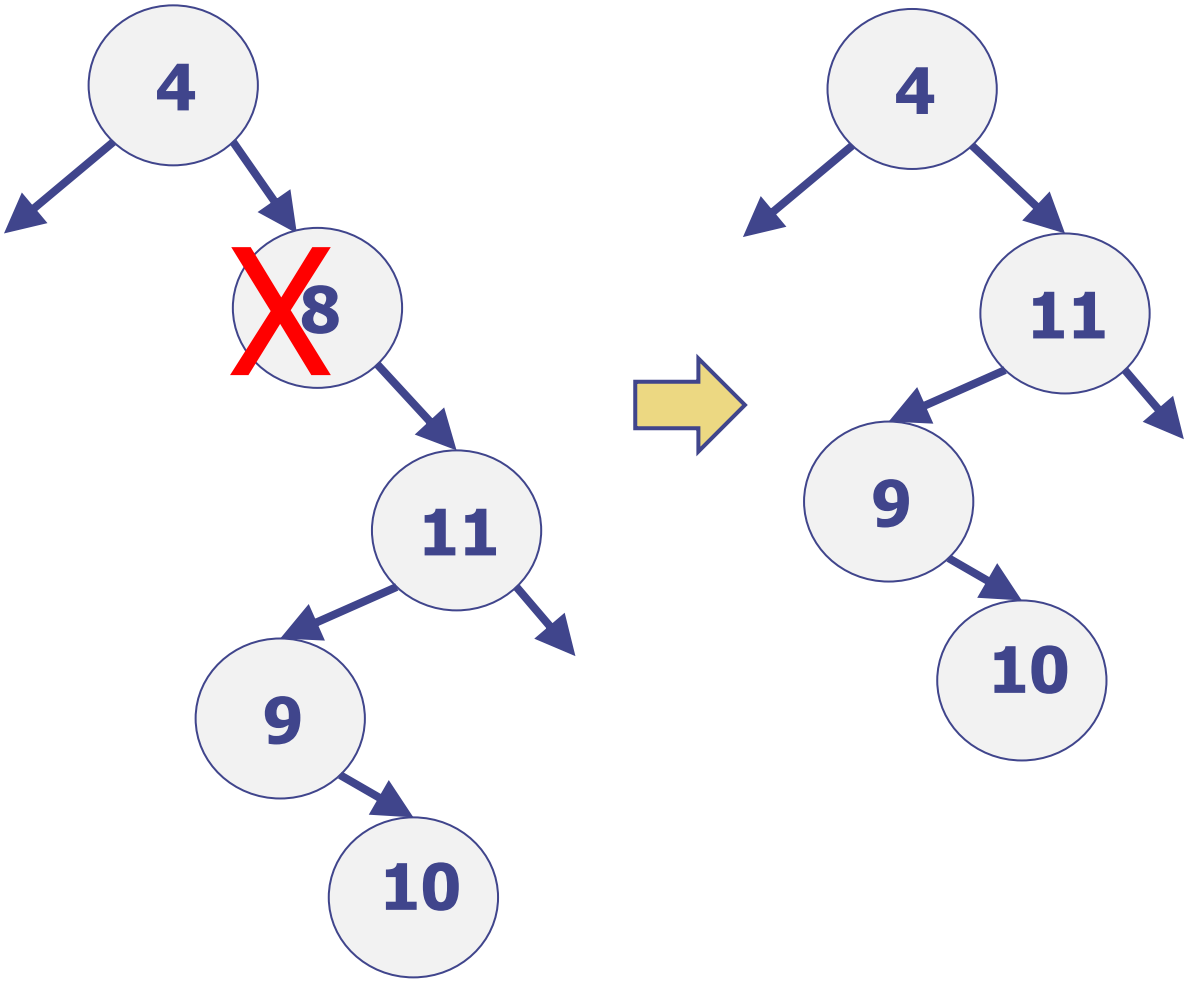
BST deletion : delete(k) : one child

- Consider delete(8)
  - Only has a left child, 7
  - has parent, 4
- From the BST properties
  - The entire sub-tree from 7 has must have keys that are >4 (and <8)
  - Hence, the subtree(7) would be allowed to be the right child of 4
  - Hence, we can remove 8, and connect 7 directly as the right child of 4



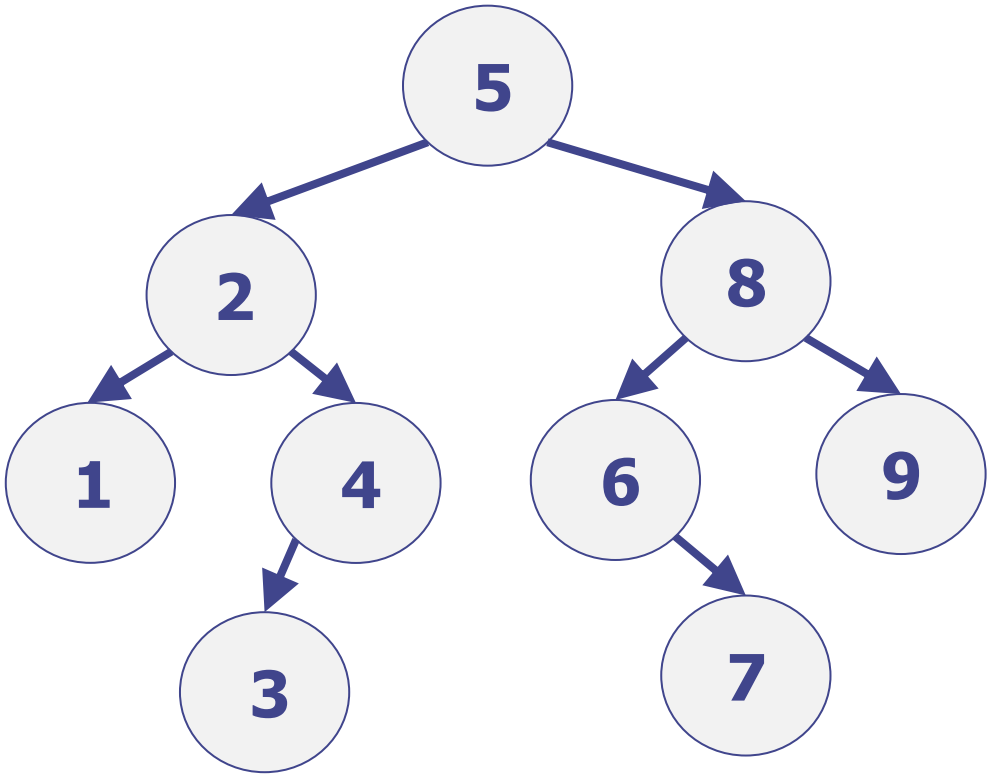
BST deletion : delete(k) : one child

- Same works if 8 has only a right child:



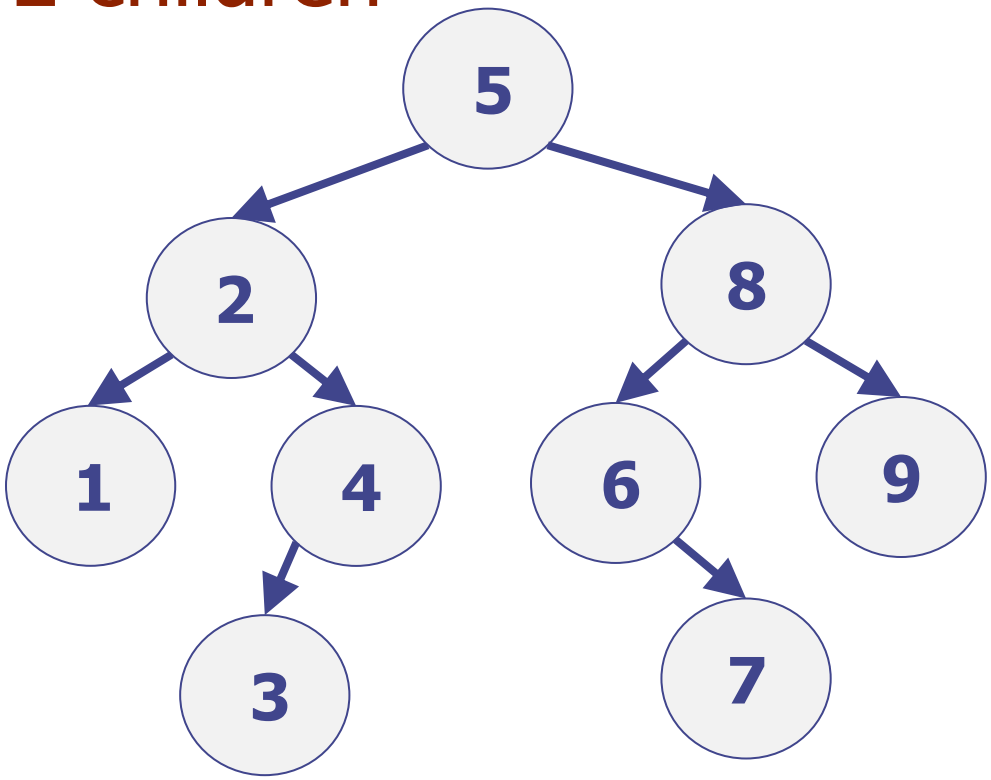
Delete with 2 children

- How could we delete(5) in this BST !?



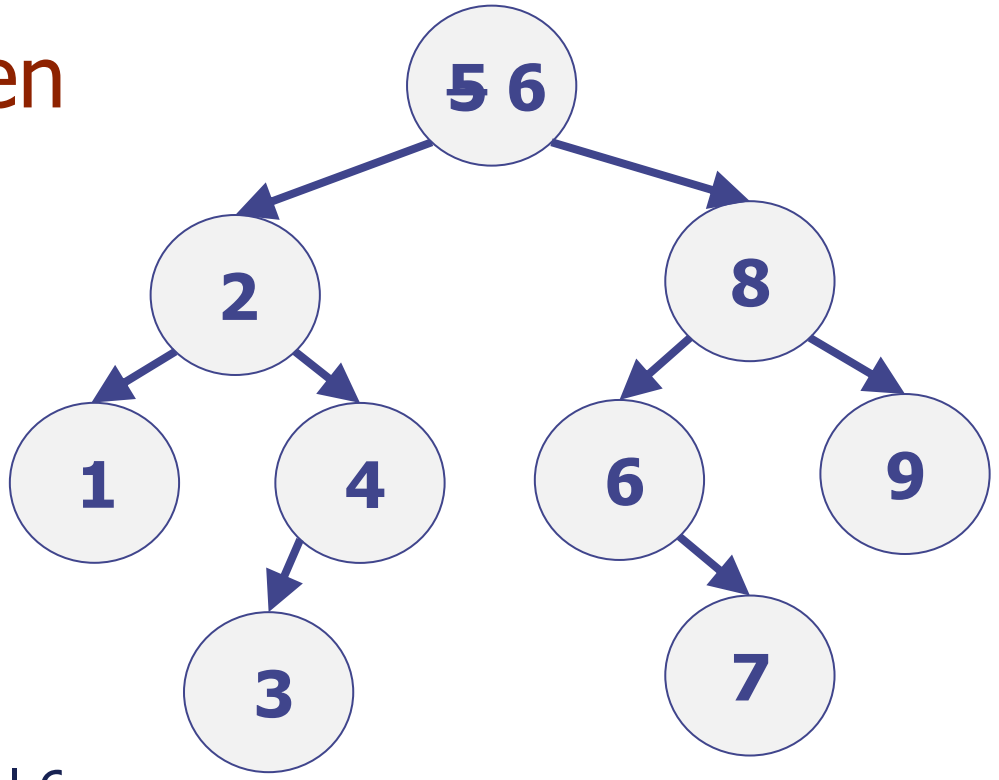
- We cannot just remove the node 5, because need to keep 2 and 8 in the same tree
- What can we do?
- ANS: Think of BST as capturing an "in-order traversal being sorted"

Delete with 2 children



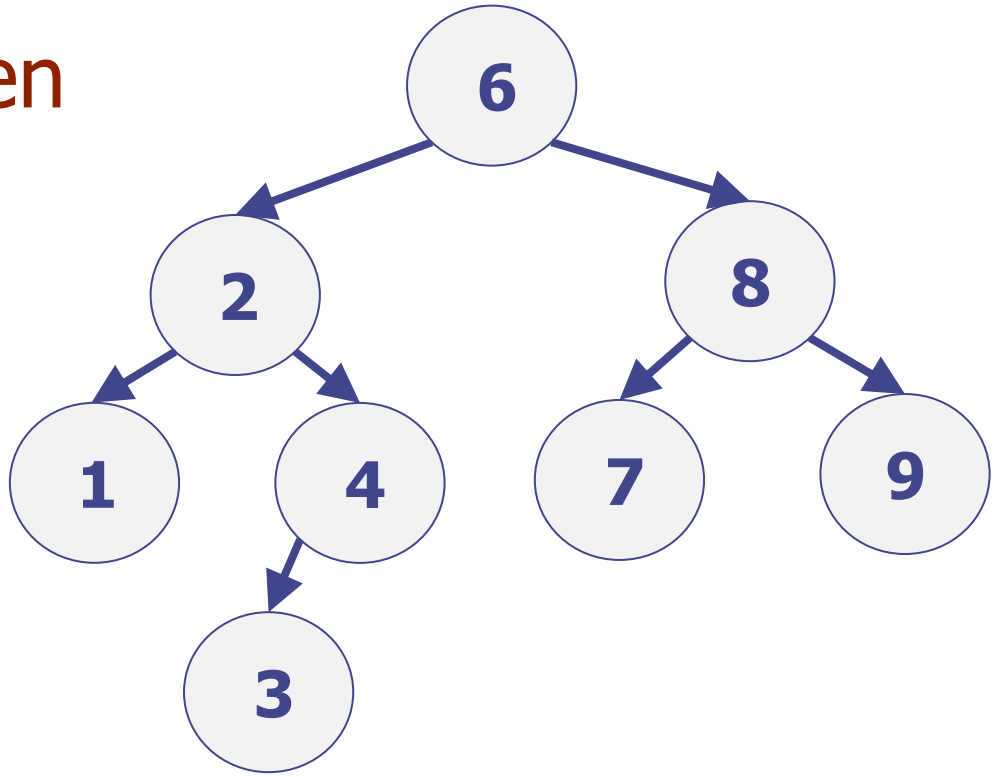
- In-order traversal give … 4,5,6, …
- After deleting 5, the in-order traversal must give … 4,6, …
- This suggests:
  - It is useful to think about involving the node 6
  - Move 6 to be after 4 in the in-order traversal
  - Copy it to overwrite 5
  - Then delete 6 instead

Delete with 2 children



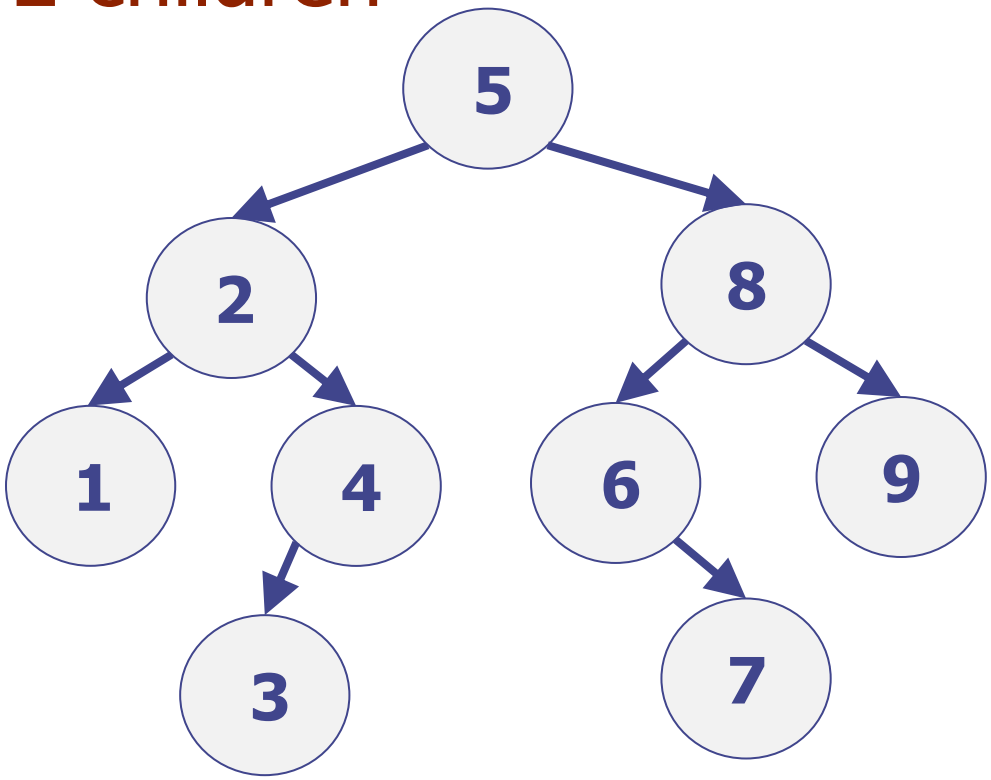
- Copied 6 to overwrite 5
  - this removes 5
- Now need to delete the original 6
- If 6 had two children we will not have made any real progress
- But node 6 cannot have 2 children - it cannot have a left child
- If 6 had a left child it would have been <6 and >5, and so would not have been
  "the node following 5 in an in-order traversal"
- So, the selected node has at most one child, and can follow previous deletion methods

Delete with 2 children



- Shows the final tree
- Note it is still a BST

Delete with 2 children



- Exercise (offline)

  Given a node, how can we efficiently find the node that follows it in an in-order traversal

  I.e. only using O(h)

Deletion with 2 children

- Suppose we want to delete node n with two children:
  - Find node p that follows node n in an inorder traversal
  - Copy the contents of p into node n
  - Delete the node p, using the deletion system for nodes with 0 or 1 child
- Exercise: convince yourself that this is O(h) not O(n)

Exercises (offline)

In the "2 children deletion", we had "we find the node that follow it in an in-order traversal"

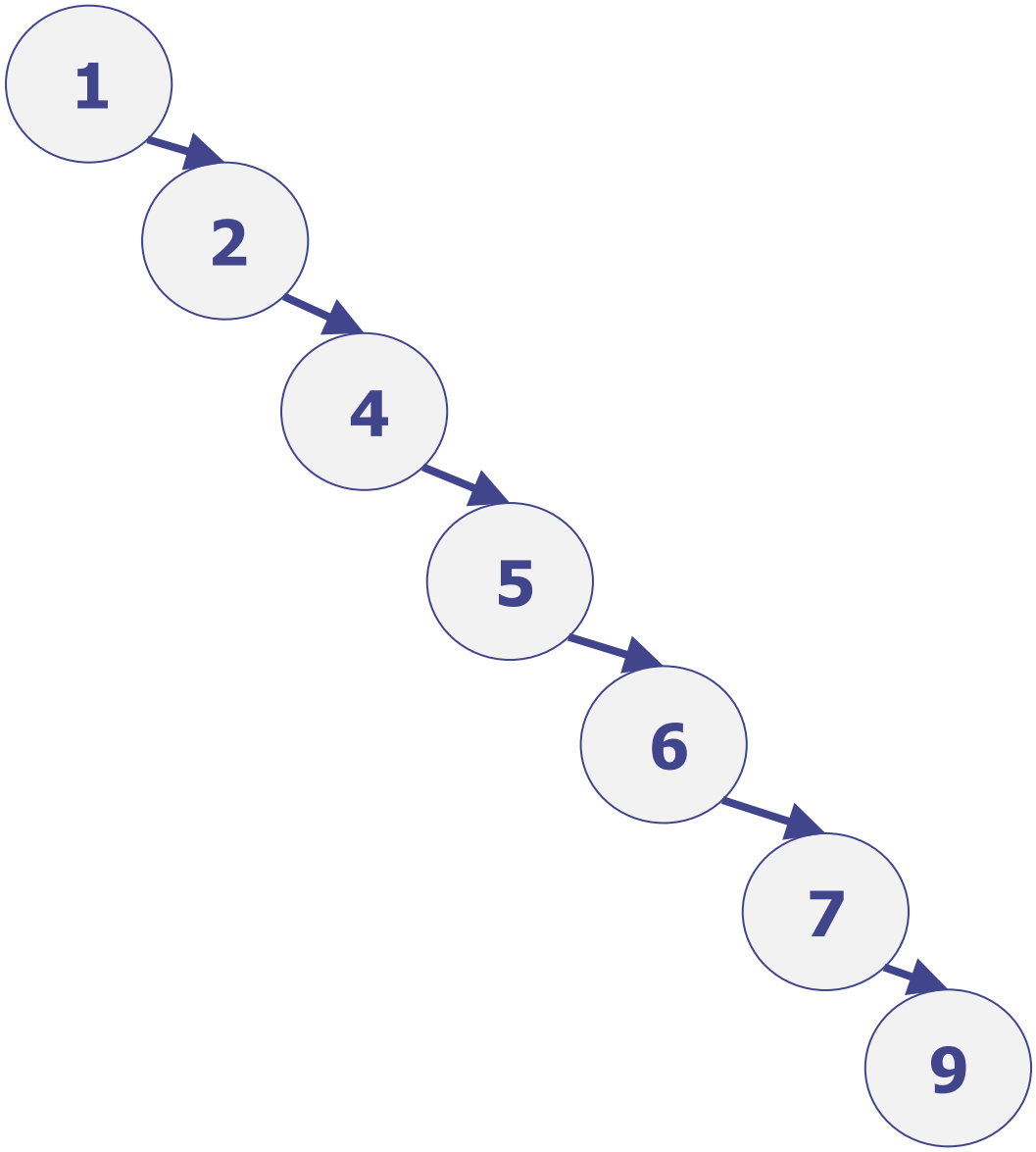
1. Could it also have been "precedes" instead of "follows"?
2. Might making a good choice of "follows" of "precedes" have some advantages?

BST Performance

- All the operations involve just moving up and down a path (or a few paths) between roots and some leaf
  - Exercise: Check this!
- Hence, all operations are O(h)
- The catch is that the height itself might not be good
  - Best case, h is O(log n)
  - Worst case, h is O(n)

Balanced and Imbalance

- Suppose we have a sorted list/array and scan it inserting entries into a BST
- New entries always go to the right child and we get a long "right-child chain" (see example)
- In such a case the height is n-1
  - Theta(n)
- The tree is said to be "imbalanced"
- The cost of the operations degenerates to O(n) rather than O(log n)

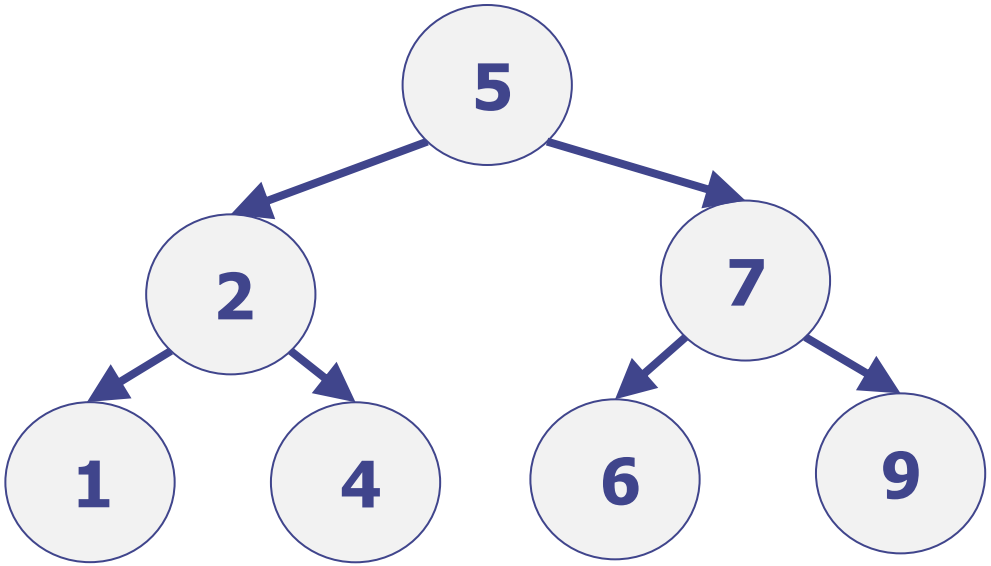


Re-balance

- We can "easily" re-balance a tree
- Use in-order traversal to extract a sorted list of keys
- Insert the median, and then recurse on the entries left of median to insert in the left subtree, and same for the right.
- Roughly

  insert(int[] A)
  - Insert the median
  - On left insert(A.leftOfMedian)
  - On right insert(A.rightOfMedian)

- Optimally reduces the height to O(log n)
  - Note the similarity to "best partitioning" in quicksort



Solution: Self-Balancing

Goal of "Self-Balancing":

- Constantly re-structure the trees:
- Keep the trees height balanced so that the height is logarithmic in the size
  - Keep h is O(log n)
- Then performance always logarithmic O(log n)

Issues in Self-Balancing

- Suppose you have a very imbalanced search tree, there are always corresponding balanced search trees
- We saw that could make trees balanced using a "total rebuild"
  - But would require O(n), and so very inefficient compared to the desired O(log n)
- Re-balancing itself needs to be O(log n) or O(height)
  - Suggests re-balancing needs to just look at the path to some recently changed node, not the entire tree
  - A priori, it is not at all obvious that this is possible!

- In a later lecture we show how to use "rotations" to adjust the tree whilst preserving the ordering
  - (but not cover all the topic of re-balancing)

Sorting using a (balanced) BST

- Insert the elements one at a time
  - Each insertion is O(h)
  - Have n insertions
  - Worst case is O(n h)
- Extract elements in sorted order by doing an in-order traversal
  - O(n)
- Hence, if we can keep height h to be O(log n) we get an O(n log n) sorting method
- https://en.wikipedia.org/wiki/Tree_sort
- (But tends to have too much overhead.)

Minimal Expectations

- The definitions of Map ADT, BST, etc
- How to use binary search trees
- The complexity of operations as function of nodes *n* and height *h*

Appendix (offline only)

- Notes that are not intended to be assessed, but that might help with how to think about tree/graph algorithms

Note to help with understanding: "Global vs. Local Perspectives"

(intended to help with coding/understanding algorithms)

- Global view of tree (or any other data structure)
  - can look at entire tree in one go
  - human seeing a picture of a (small) tree
- Local view of tree
  - what your code sees: "code perspective"
  - code generally only sees a local portion of a tree and must work with that
- Vital coding skill: develop ability to see the local view, "code perspective", and not only the global
  - at each code line need to know which data is immediately accessible
  - Avoid trying to code in a way that cannot be done locally
  - Note the local view still needs to work towards a global goal

Note intended to help with your coding: "Global vs. Local Perspectives" Analogies

- Your Data Structure:
  - A real tree
- Your code:
  - "A short-sighted ant crawling over the tree"

Note intended to help with your coding: "Global vs. Local Perspectives" Analogies

- Your Data Structure:
  - A big underground cave system
- Your code:
  - You!
  - … with a weak flashlight
  - … and no overall map
  - … but a goal of which cavern to reach

Note intended to help with your coding: "Global vs. Local Perspectives" Analogies

- Your Data Structure:
  - The entire web
- Your code:
  - You!
  - … can only follow links
  - … no search engine – no global view
  - … but a goal of finding a webpage with a given phrase